



Modern Coding and advanced detection report

Performed by:

Georgii BARBER

Yahya YOUNES

3A-SATCOM

2023-2024

Contents

1	Introduction	3
2	Repetition and parity-check codes	3
2.1	Question 1	3
2.1.1	Associated parity matrix	3
2.1.2	Verifying duality of codes	4
2.2	Question 2	4
2.2.1	Deriving the ML decoder in the probability domain for a binaryinput	4
2.3	Question 3: Decoders in the LLR domain	5
2.4	Question 4: Expression of the bit MAP decoder	5
2.5	Question 5: Deriving the bit MAP decoder in the logarithmic domain	6
3	ML and MAP decoding of an Hamming code	7
3.1	ML decoding by cross-correlation	8
3.2	MAP bit decoding by direct calculation	8
4	LDPC Simulations	11
5	Turbo codes	13
6	Conclusion	22

1 Introduction

In telecommunications networks, the modulation coding scheme significantly influences key characteristics such as reliability, data transmission rate, and error correction. Efficient modulation coding involves utilizing error correction codes to organize bits into symbols for modulation and transmission. Selecting the appropriate error correction code is crucial for maximizing efficiency and conserving resources like time and frequency. The field offers various error correction codes and decoding methods, enhancing system flexibility. Notable decoding methods include Maximum Likelihood (ML) and Maximum A Posteriori (MAP) decoding. Forward Error Correction (FEC) codes, like Turbo and Low-Density Parity Check (LDPC) codes, along with Error Detection Codes (EDC) such as checksums, provide robust error correction capabilities.

2 Repetition and parity-check codes

2.1 Question 1

2.1.1 Associated parity matrix

Both of these codes fall under the category of systematic linear block codes. The construction of parity check matrices is such that they are formulated as an amalgamation of the identity matrix and additional parity columns.

Consider the Repetition code $C_1(N, 1)$:

$$G_1 = [\underbrace{1 \dots 1}_N] = [I_1 \quad P_1]$$

where

$$P_1 = [\underbrace{1 \dots 1}_{N-1}]$$

The parity matrix H_1 , for which G_1 .

$$H_1 = [P_1^\top \quad I_{N-1}] = \begin{bmatrix} 1 & & \\ \vdots & & \\ 1 & & I_{N-1} \end{bmatrix}$$

In the context of the parity-check code $C_2(N, N - 1)$: We can similarly deduce that:

$$H_2 = [P_2^\top \quad I_1] = [\underbrace{1 \dots 1}_N]$$

With

$$P_2 = \begin{bmatrix} 1 \\ \vdots \\ 1 \end{bmatrix}_N = P_1^\top$$

2.1.2 Verifying duality of codes

To establish the duality of the codes $C_1(N, 1)$ and $C_2(N, N - 1)$, we need to confirm:

$$G_2 \cdot G_1^\top = 0$$

The proof is as follows:

$$\begin{aligned} G_2 \cdot G_1^\top &= G_2 \cdot [I_1 \quad P_1]^\top \\ &= G_2 \cdot [I_1 \quad P_2^\top] \\ &= G_2 \cdot H_2^\top \\ &= 0 \end{aligned}$$

Therefore, $C_1(N, 1)$ is the dual of $C_2(N, N - 1)$. By symmetry, it follows that $C_1(N, 1)$ and $C_2(N, N - 1)$ are dual codes.

2.2 Question 2

2.2.1 Deriving the ML decoder in the probability domain for a binary input

Let us consider an input u being encoded into c for the Repetition code $C(N, 1)$, where $u = u_0$ and $c = (u_0, \dots, u_0)$. The received vector y is $y = (y_0, \dots, y_{N-1})$, and the estimate $\hat{c}$ is $\hat{c} = (\hat{c}_0, \dots, \hat{c}_{N-1})$. In the Maximum Likelihood (ML) decoding approach, $\hat{c}$ is determined as the argument that maximizes $P(y|c')$.

For the Repetition code, we consider only two possible codewords, $c' = (0, \dots, 0)$ or $c' = (1, \dots, 1)$. Therefore, a single threshold is necessary for decoding:

$$\hat{u} = \begin{cases} 0 & \text{if } P(y|c' = (0, \dots, 0)) > P(y|c' = (1, \dots, 1)), \\ 1 & \text{if } P(y|c' = (0, \dots, 0)) \leq P(y|c' = (1, \dots, 1)) \end{cases}$$

This process is in the probability domain. In the domain of Log-Likelihood Ratios (LLR):

$$\hat{u} = \frac{1 - \text{sign}(LLR_{ML})}{2}$$

With the parity check code framework, the encoding process takes an input u through $C(N, N - 1)$ to produce c , where $u = (u_0, \dots, u_{N-2})$, and $c = (c_0, \dots, c_{N-1})$, with c_{N-1} being the calculated parity bit as $u_0 \oplus \dots \oplus u_{N-2}$. Upon transmission, the received sequence y is $y = (y_0, \dots, y_{N-1})$, with the estimated codeword being $\hat{c} = (\hat{c}_0, \dots, \hat{c}_{N-1})$.

The Maximum Likelihood (ML) decoding method aims to find $\hat{c}$ that is the argument of the maximal probability $P(y|c')$. With 2^{N-1} potential codewords and the first $N - 1$ bits corresponding to the information word, the parity check code is demonstrated to be

systematic. Therefore, $2^{N-1} - 1$ thresholds are required for decoding and recovery of the estimated codewords. Consider c_0 as an example of an estimated codeword.

$$\hat{c} = c_0 \quad \text{if} \quad \begin{cases} P(y|c_0) > P(y|c_1) \\ P(y|c_0) > P(y|c_{2N-1-1}) \end{cases} \quad \{c_0, \dots, c_{2N-1-1}\} \in \mathcal{C}$$

in the probability domain

To decode and accurately estimate each codeword, it's necessary to evaluate all potential codewords against the received word. Under the assumption of a memoryless channel and independent transmissions, we consider the estimation of c_0 as $(bc_{N-2}, \dots, bc_0)$, where bc_i denotes the constituent bits of c . Verification proceeds as follows:

$$\hat{c} = c_0 \quad \text{if} \quad \begin{cases} P(by_0|bc_0) > P(by_0|\overline{bc_0}), \\ \vdots \\ P(by_{N-1}|c_{N-1}) > P(by_{N-1}|\overline{bc_{N-1}}) \end{cases}$$

where $by_0 = (by_{N-2}, \dots, by_0)$ denotes the received bit sequence.

2.3 Question 3: Decoders in the LLR domain

In the LLR domain :

$$\hat{bc}_i = \frac{1 - \text{sign}(LLR_{MLi})}{2}, \quad \text{for } i \in \{0, \dots, N-1\} \quad \text{and} \quad \hat{c} = (bc_{N-1}, \dots, bc_0)$$

with the LLR given by:

$$LLR_{MLi} = \log \left(\frac{P(by_i|bc_i = 0)}{P(by_i|bc_i = 1)} \right)$$

2.4 Question 4: Expression of the bit MAP decoder

We know that we can write the MAP formula as follows

$$\hat{c}_n = \arg \max_{c_n} P(c_n|y)$$

$$P(c_n|y) = \sum_{c' \in \mathcal{C}} P(c'_0, \dots, c'_n, \dots, c'_{N-1}|y)$$

Since $P(y)$ does not depend on c' , we can remove it, resulting in

$$\propto \sum_{c' \in \mathcal{C}} P(y|c'_0, \dots, c'_n, \dots, c'_{N-1}) \cdot P(c'_0, \dots, c'_n, \dots, c'_{N-1})$$

Assuming uniform distribution of the generated codes, we simplify to

$$\propto \sum_{c' \in \mathcal{C}} P(y|c'_0, \dots, c'_n, \dots, c'_{N-1})$$

So, we have

$$\hat{c}_n = \begin{cases} 0 & \text{if } P(c'_n = 0|y) > P(c'_n = 1|y) \\ 1 & \text{if } P(c'_n = 0|y) \leq P(c'_n = 1|y) \end{cases}$$

This is equivalent to comparing the sum of probabilities where c'_n is 0 and 1, respectively.

$$\begin{aligned} \hat{c}_n = 0 & \quad \text{if} \quad \sum_{c' \in \mathcal{C}, c'_n=0} P(y|c') > \sum_{c' \in \mathcal{C}, c'_n=1} P(y|c') \\ \hat{c}_n = 1 & \quad \text{if} \quad \sum_{c' \in \mathcal{C}, c'_n=0} P(y|c') \leq \sum_{c' \in \mathcal{C}, c'_n=1} P(y|c') \end{aligned}$$

2.5 Question 5: Deriving the bit MAP decoder in the logarithmic domain

In the LLR domain, expression (*) is equivalent to say:

$$\hat{c}_n = \frac{1 - \text{sign}(LLR_{MAP})}{2}$$

With:

$$LLR_{MAP}(\hat{c}_n) = \log \left(\frac{\sum_{c' \in \mathcal{C}, c'_n=0} P(y|c')}{\sum_{c' \in \mathcal{C}, c'_n=1} P(y|c')} \right) \quad (**)$$

Repetition code In the case of a repetition code, which has only two possible codewords, this expression can be simplified as follows:

$$LLR_{MAP}(\hat{c}_n) = \log \left(\frac{P(y|c' = (0, \dots, 0))}{P(y|c' = (1, \dots, 1))} \right)$$

Case for the Gaussian channel with binary inputs We have:

$$y = c' + w$$

where w is the Gaussian Noise, follows the Centered Normal Distribution $\mathcal{N}(0, \sigma_n^2)$ with the probability density function:

$$f_W(w_n) = \frac{1}{\sqrt{2\pi\sigma_n^2}} e^{-\frac{w_n^2}{2\sigma_n^2}}$$

The probability $P(y|c')$ is given by:

$$\begin{aligned}
P(y|c') &= P(y_{N-1}, \dots, y_0 | c'_{N-1}, \dots, c'_0) \\
&= \prod_{n=0}^{N-1} P(y_n | c'_n) \quad (\text{Memoryless Gaussian channel}) \\
&= \prod_{n=0}^{N-1} P(c_n + w_n | c'_n) \\
&= \prod_{n=0}^{N-1} P(w_n) \quad (\text{with } w_n = y_n - c_n) \\
&\propto \prod_{n=0}^{N-1} e^{-\frac{1}{2\sigma_n^2} (y_n - c_n)^2} \\
&\propto e^{-\frac{1}{2\sigma^2} \sum_{n=0}^{N-1} (y_n - c_n)^2}
\end{aligned}$$

Given that the sum of squared received symbols $\sum_{n=0}^{N-1} |y_n|^2$ is independent of the estimated codewords and the sum of squared codeword symbols $\sum_{n=0}^{N-1} |c_n|^2$ does not convey much information about the codewords we aim to estimate (for instance, with a PSK mapping, this is a constant), we can focus on the terms that contain information about the probability of correctly reproducing y given c' as $-\sum_{n=0}^{N-1} (y_n \cdot c_n)$. Thus, we have:

$$\begin{aligned}
P(y|c') &\propto \exp \left(-\frac{\sum_{n=0}^{N-1} |y_n - c_n|^2}{2\sigma^2} \right) \\
&\propto \exp \left(-\frac{\sum_{n=0}^{N-1} (y_n \cdot c_n)}{2\sigma^2} \right)
\end{aligned}$$

The $LLR_{MAP}(c_n)$ in (**) can be rewritten as:

$$LLR_{MAP}(c_n) = \log \left(\frac{\sum_{c' \in \mathcal{C}, c'_n=0} \exp \left(-\frac{\sum_{i=0}^{N-1} (y_i \cdot c_i)}{2\sigma^2} \right)}{\sum_{c' \in \mathcal{C}, c'_n=1} \exp \left(-\frac{\sum_{i=0}^{N-1} (y_i \cdot c_i)}{2\sigma^2} \right)} \right) \quad (***)$$

3 ML and MAP decoding of an Hamming code

In this section, we aim to implement various types of linear encoders to compare ML (Maximum Likelihood) and MAP (Maximum A Posteriori) decoding techniques. Our communications scheme is as follows:

$$u \xrightarrow{\text{Encoder}} c \xrightarrow{\text{AWGN channel}} y \xrightarrow{\text{Decoder}} \hat{c}$$

We will begin with a Parity-Check encoder, followed by a Repetition encoder and a Hamming encoder. The decoding will be performed using either MAP or ML decoders.

For MAP decoding, we will apply the rule specified in the LLR (Log-Likelihood Ratio) domain for a Gaussian channel, as detailed in expression above of the report. For ML decoding, we will measure the correlation between the received word and all possible emitted codewords, selecting the one with the highest correlation to the received word.

For these three types of encoders, the underlying code structure will remain unchanged; only the generator matrix will be altered for each case. However, for the Parity-Check encoder, we will take advantage of its structure to implement a Belief Propagation (BP) approach for MAP decoding.

3.1 ML decoding by cross-correlation

The MATLAB function that performs the ML decoding of a Hamming code using the correlation method is:

[frame=single,style=Matlab-Pyglke] `hammingmlaecoding.mreceivedcodeword H` is the parity check matrix of the code.

3.2 MAP bit decoding by direct calculation

The MATLAB function that performs the MAP decoding of a Hamming code using the direct calculation method is:

[frame=single,style=Matlab-Pyglke] `hammingmapaecoding.mreceivedcodeword H` is the parity check matrix of the code and p_0 is the prior probability of a bit being 0.

Consider the example of a Parity Check code $C(4, 3)$. The Tanner graph for this code is depicted as follows:

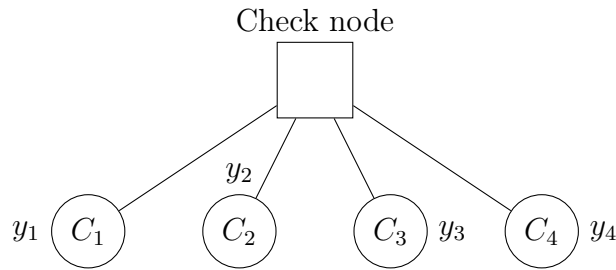


Figure 1: Tanner graph for Parity Check code $C(4, 3)$

Each variable node computes its log-likelihood ratio (LLR) using the propagated information on the graph. The LLR for a variable node c_i , denoted by $\ell_{\text{tr}}(c_i)$, is the sum of the LLR from the channel observation $\ell_{\text{ch}}(y_i)$ and the LLR from other variable nodes $\ell_{\text{extr}}(c_i)$, which is given by:

$$\ell_{\text{tr}}(c_i) = \ell_{\text{ch}}(y_i) + \ell_{\text{extr}}(c_i)$$

where $\ell_{\text{extr}}(c_i)$ is calculated as:

$$\ell_{\text{extr}}(c_i) = 2 \cdot \tanh^{-1} \left(\prod_{j=1, j \neq i}^4 \tanh \left(\frac{\ell_{\text{tr}}(c_j)}{2} \right) \right)$$

For this graph, a single iteration is sufficient to estimate the codeword.

Parity check code

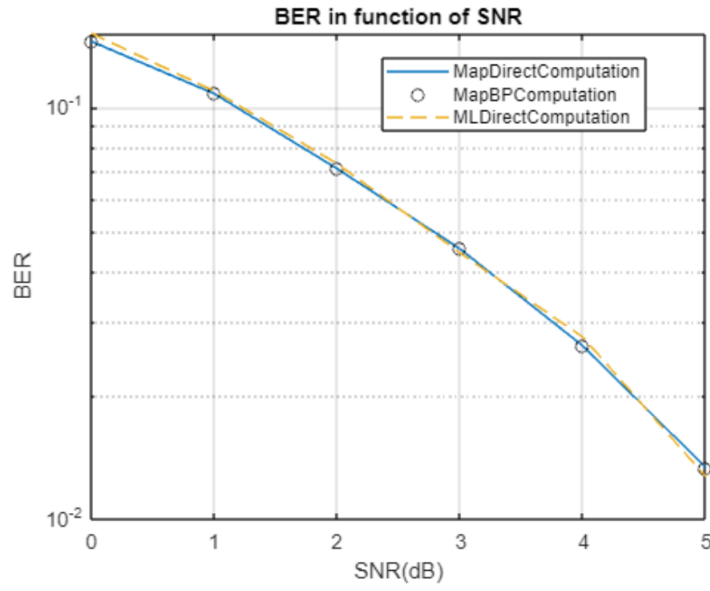


Figure 2: Parity Check code : BER in function of E_b/N_0

Repetition code

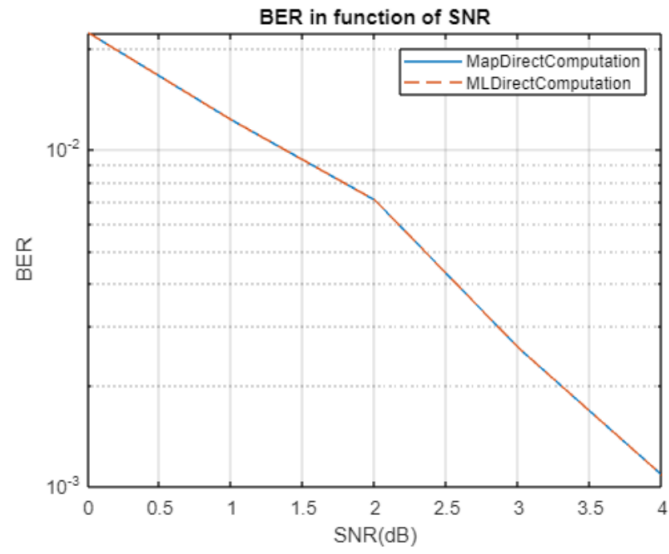


Figure 3: Repetition code : BER in function of E_b/N_0

Hamming code

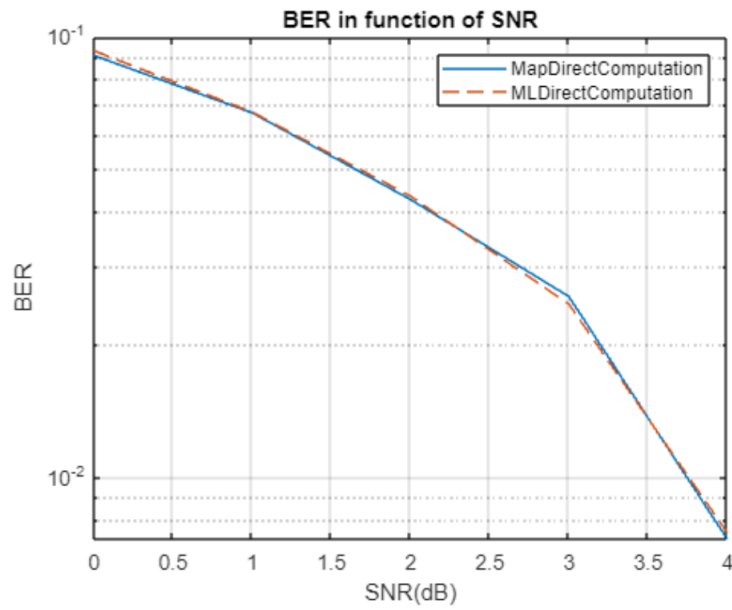


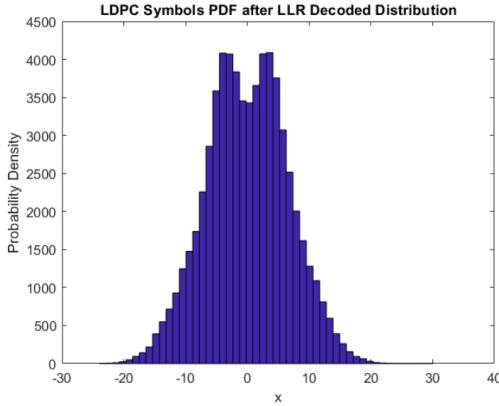
Figure 4: Hamming code : BER in function of E_b/N_0

4 LDPC Simulations

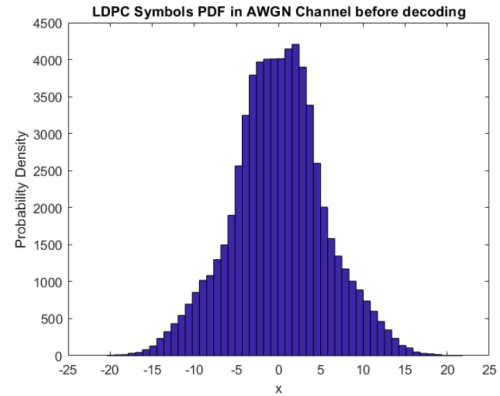
The initially provided code “testldpcgaussien” is an LDPC simulation through an AWGN channel. The code itself uses the function “myBPdecoder” and a fixed set of SNR values to iterate the LDPC decoding with the LLR method.

The chosen SNR values range from 4 to 100 dB, specifically $\text{SNRrange} = [2, 4, 10, 30, 100]$. Through these SNR ranges and within the inner loops of the code, the iterative nature of LDPC can be discerned. Indeed, the higher the SNR, the lower the number of iterations needed as many of our matrices have fewer parity checks to make and therefore less calculations need to be made.

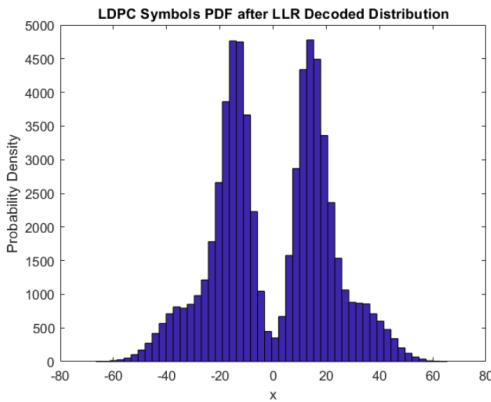
An example of the difference between a start of iteration and end of iteration can be seen in the figures below (4dB SNR, Iter No1 vs 4dB SNR, Iter No6):



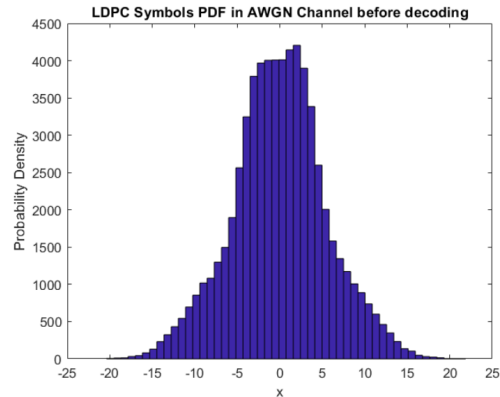
(a) Iteration No 1 – SNR = 4dB



(b) Post AWGN Channel PDF – SNR = 4dB



(c) Iteration No6 - SNR = 4dB



(d) Post AWGN Channel PDF - SNR = 4dB

Figure 5: LDPC Simulation Results

We can see that initially (Figure 5a: Iteration No 1 – SNR = 4dB), the histogram which represents the LLR and therefore the confidence of the belief propagation (BP) de-

coding algorithm is very similar to the pre-BP channel. This is normal as LDPC require several iterations to achieve optimal decoding confidence. The higher the number of iterations, the greater the confidence in the decoding of the symbols $+1$ and -1 . This can be shown in iteration No 6 of the same channel (Figure 5c: Iteration No6 – SNR = 4dB) still, there remains some ambiguity around 0 even in Iter No 6.

That ambiguity disappears once iteration 10 is reached, with the associated PDF displayed below

By pausing the graph that briefly appears after the “spy” (which plots the sparsity pattern of the matrix - Figure ??) function it can be noted that the generated parity-check matrix is very dense in the upper left portion (non-zero elements = 226799). The matrix is also quite sparse and triangular around the bottom right, which gives the decoder more speed at the expense of less coding complexity. This type of parity-check matrix resembles the DVB-S2 standard parity check matrix.

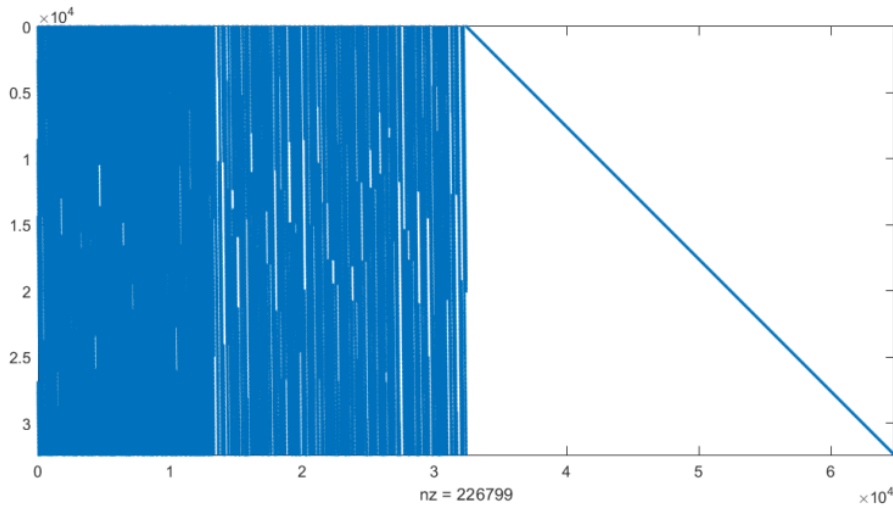


Figure 6: Parity-Check Matrix

By running the LDPC script with several SNR other than 4dB (`SNRrange = [2,4,10,30,100]`), a significant speed and performance improvement can be noted. Although the “perf” values remain the same in the end for any of the aforementioned SNR values (all around 0.5 ± 0.0008) because a satisfactory symbol error rate is reached before the loop forcibly ends, the amount of iteration performed at each SNR level varies.

The LDPC script and its parity-check matrix aren’t able to handle a 2dB SNR on an AWGN channel. This is clear because the maximum number of iterations is reached without a clear separation of the symbols in the resulting PDFs (We can therefore assume that the decoding threshold is somewhere between 2dB and 4dB for this specific LDPC).

SNRrange (dB)	Iterations needed (approx.)
2dB	50 (Max)
4dB	10
10dB	2
30dB	1
100dB	1

Table 1: Number of iterations needed for different SNR values.

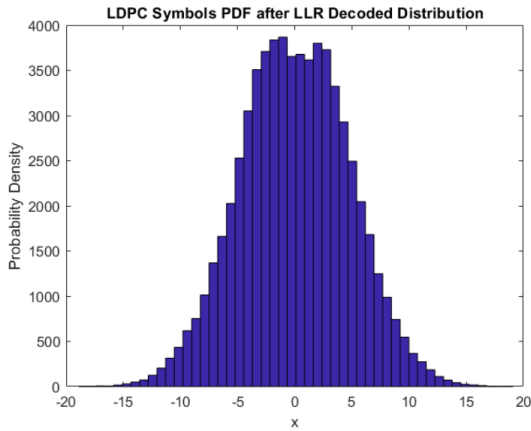


Figure 7: Iteration No 35 – SNR=2dB

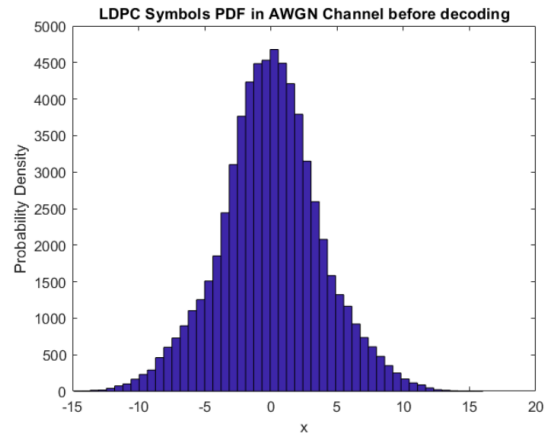


Figure 8: Post AWGN Channel PDF- SNR=2dB

5 Turbo codes

The code provided executes a parallel Turbo Code (constraint length = 5, Generator Polynomials $[37, 21]_8$) through an AWGN channel with modifiable parameters (Modulation parameter, Info Frame size, Nb Frames, EbNo range etc.)

Turbo Codes are designed in a way which, after each iteration, gets the results closer and closer to the Shannon capacity limit. This code uses a parallel architecture (Figure ??) with an interleaver between the two parity streams (with a total of three streams, a relatively classic architecture seen in class and in the exam). This in turn, means that the more iterations we have, the better the results.

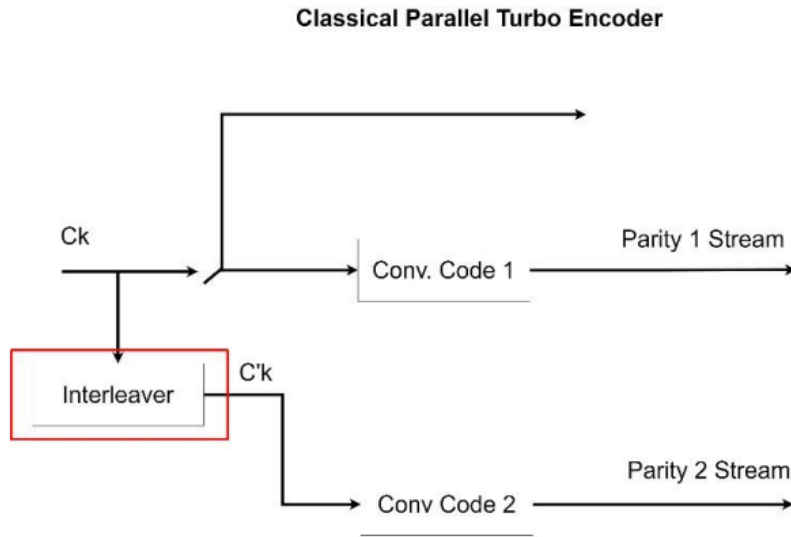


Figure 9: Parallel Turbo Code (Encoder) General Architecture

One of the most interesting things which can be modified, and which can have a significant impact on the final result, is the interleaver. Indeed, the complexity, type and randomness or lack thereof of the interleaving vector/matrix can have significant effects on Turbo Codes (and our Turbo Code example in particular). In this Turbo Code section, we shall explore different results that come with changing the interleaver properties

A first 3-Iteration run with a random interleaver (mt19937ar) produces the following results:

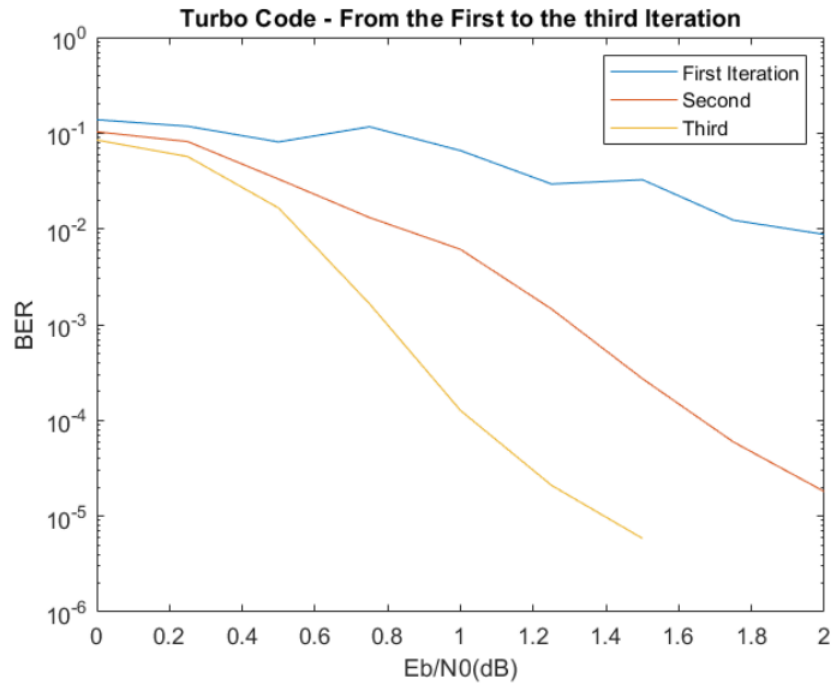


Figure 10: Turbo-Code, Rand Interleaver (mt19937), frame size = 4096, Seed Number 11

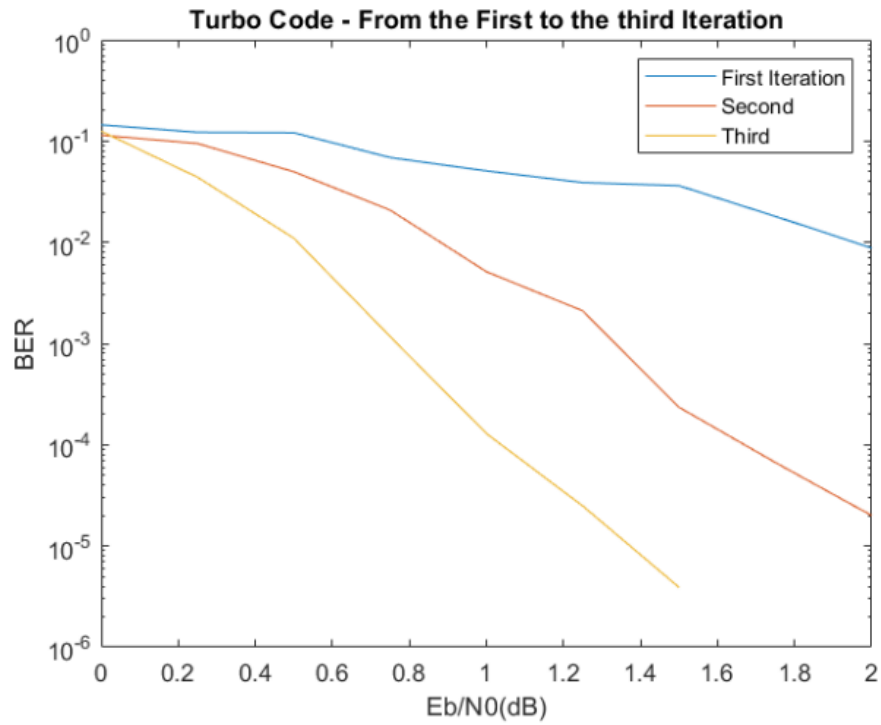


Figure 11: Turbo Code, Rand Interleaver (mt19937), frame size = 4096, Seed Number 100

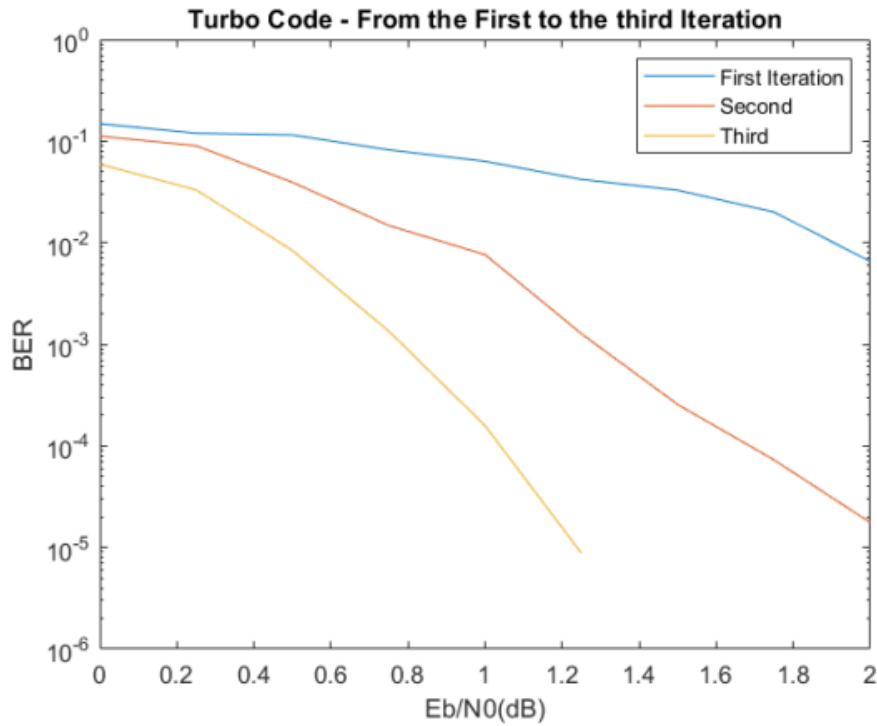


Figure 12: Turbo Code, Rand Interleaver (mt19937), frame size = 4096, Seed Number 1

Seed number does not seem to affect the Turbo Code performance significantly. However, the main issue which can arise with poorly designed interleavers is the potential correlation introduced by it. This correlation ties together the two parallel streams (Figure 13), which impacts the diversity and the BER floor of the code in a negative way

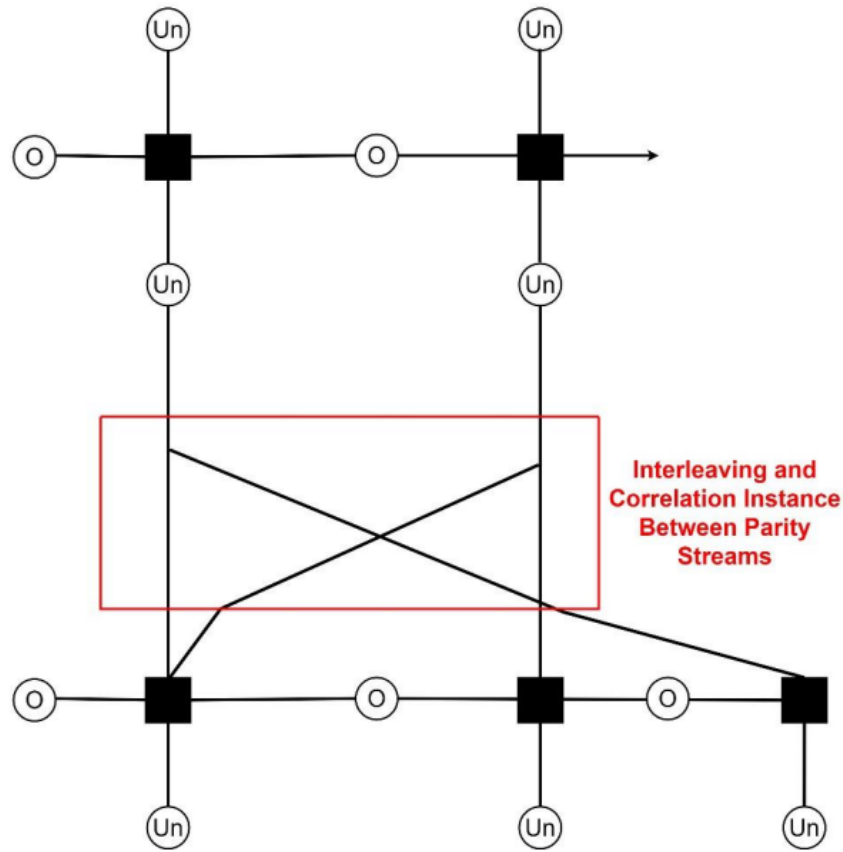


Figure 13: Correlation Between Streams Due To Interleaver

This can be observed in the figures below, the first of which carries no random permutation, so the correlation is quite high:

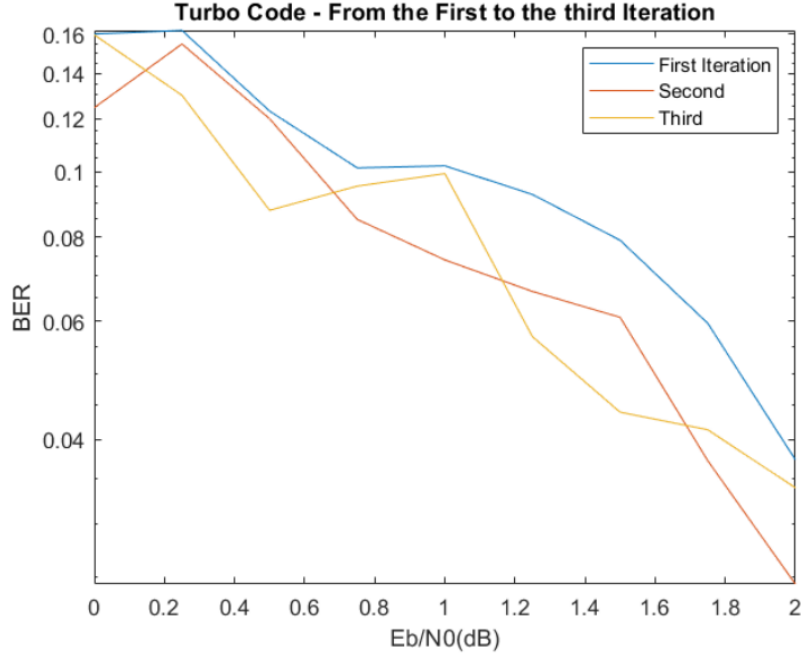


Figure 14: Turbo Code, Rand Interleaver (mt19937) – no Rand permutation, frame size = 4096, Seed Number 1

The results of the previous simulation (Figure ??) are significantly worse than the previous results. So much so in fact that we don't even reach a BER floor due to the poor performance of the code. Another type of trivial interleaver is just a cyclic shifter by a constant (10 in our case), this kind of interleaver has similar results to the one in Figure ??, whatever the iteration, the result is the same (and bad).

An attempt was made to explore the effects of changing the type of random stream generator, for example instead of using 'mt19937ar', the 'shr3cong' generator was used (different obscure properties which don't need to be explored here).

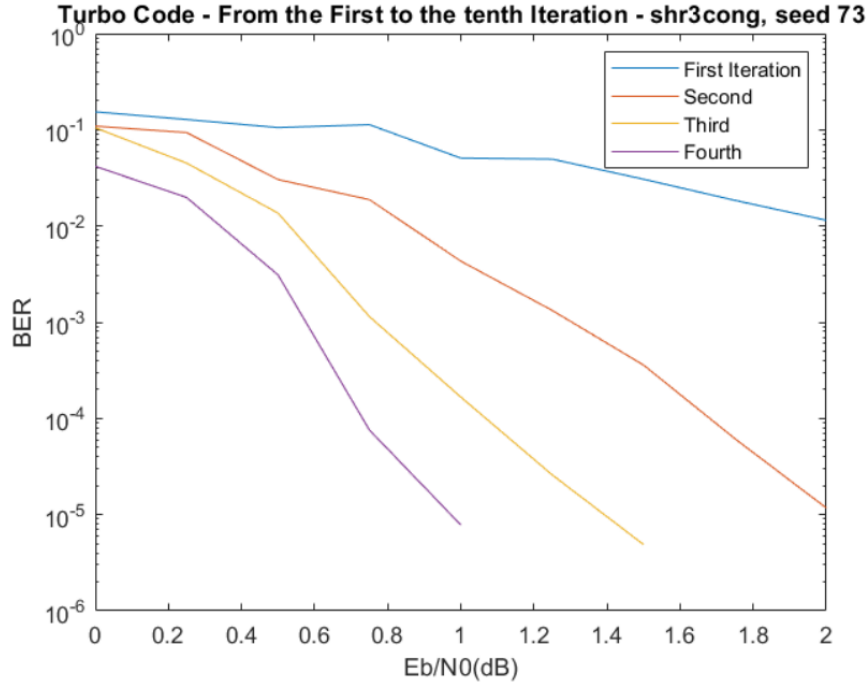


Figure 15: Turbo Code, Rand Interleaver (Cong, seed = 73) and Permutator, frame size = 4096

Although no significant change was visible on Figure 15 compared to other types of random stream generators seen in Figure 12 and Figure 11, significant plateaus were reached when using a 64×64 matrix interleaver (`intrlvIndices = matintrlv(1:K, Nrows, Ncols);`). The plateaus observed at the bottom right of Figure 16 are indicative of the limitations introduced by a simple matrix interleaver rather than a random one. The distance and correlation of the codewords has reached a point where running more iterations does not add anything notable in terms of BER performance. The correlation that can occur between streams can be visualized in the simplified sketch on , in which a typical parallel Turbo Code architecture can be seen as well.

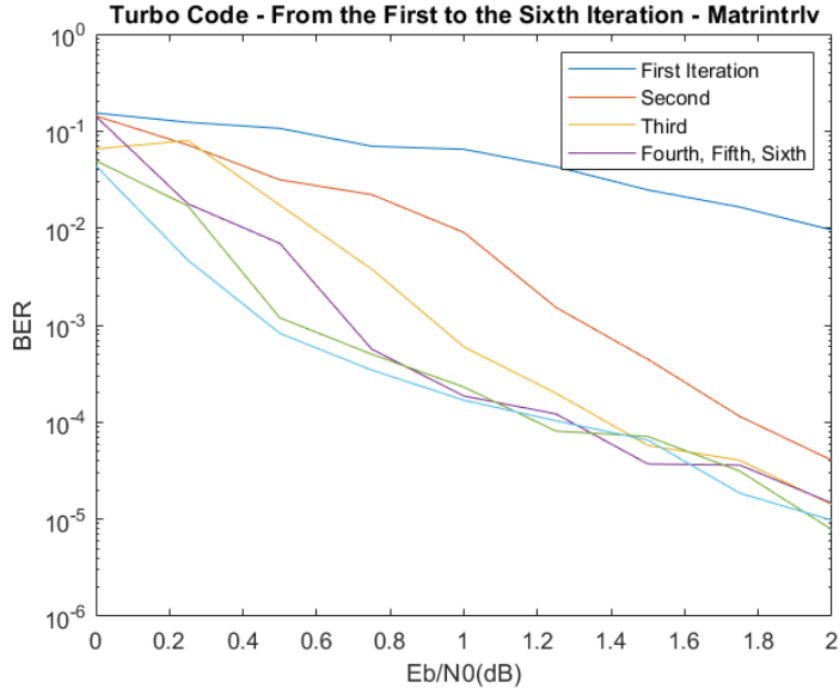


Figure 16: Turbo Code, Rand stream, 64 x 64 matrix interleaver, frame size = 4096

Something similar can be said when instead of using a 64 x 64 matrix, a 16 x 256 matrix was chosen. The performance is slightly worse than the square matrix.

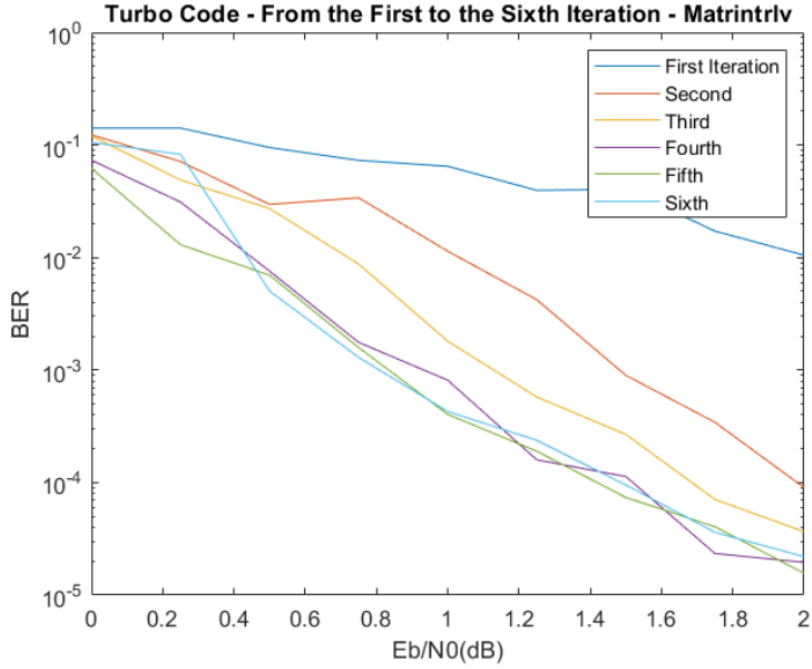


Figure 17: Turbo Code, Rand stream, 16 x 256 matrix interleaver, frame size = 4096

This leads us into the very niche but valuable study of the optimal interleaver per application. There are no general formulas which outline the best type and optimal row x columns ratio per Turbo Code, but educated guesses can be made by running preliminary simulations. Should we want to proceed with research into which interleaver is best, then definitely starting with a random interleaver is the best option given its superior performance (as seen below in Figure 18).

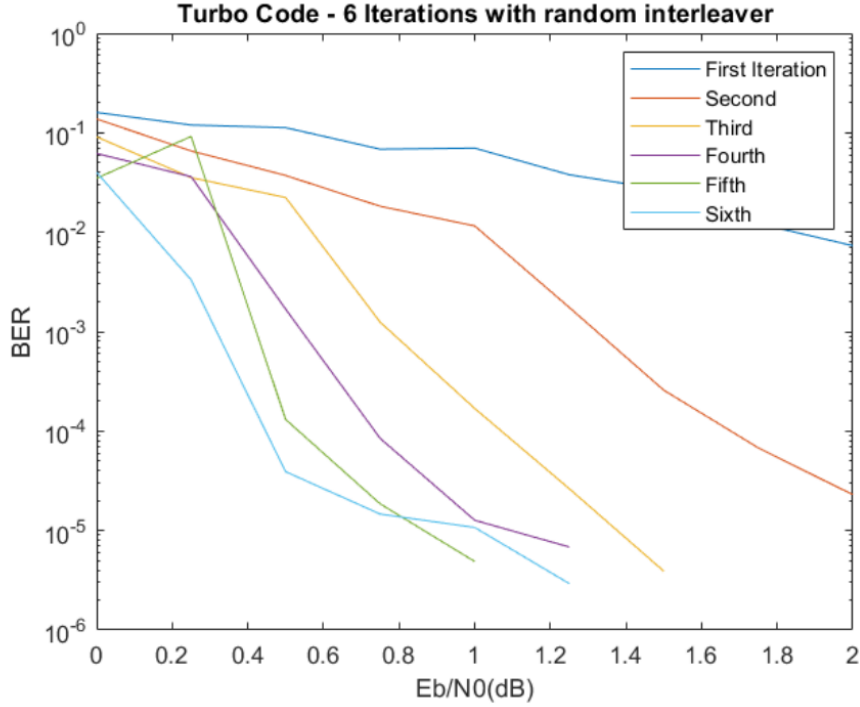


Figure 18: Turbo Code, Rand stream, Random interleaver, frame size = 4096

6 Conclusion

In conclusion, the performance of Turbo Codes is highly contingent upon several critical factors. When utilized within an appropriate E_b/N_0 environment, these codes are capable of approaching the Shannon limit, offering near-optimal error correction capabilities. The number of iterations plays a pivotal role; too few, and the error correction is insufficient, too many, and the system becomes computationally burdensome, diminishing the returns in terms of error reduction. It is thus crucial to strike a balance, optimizing the number of iterations to match the specific requirements of the communication channel and the desired quality of service.

Furthermore, the choice of interleaver can dramatically affect performance. A well-designed interleaver will ensure that the codewords are sufficiently randomized, thereby reducing the likelihood of error floors and enhancing the overall robustness of the communication system. This selection is highly context-dependent, varying with the channel conditions and the specific application at hand. For instance, channels with more pronounced burst errors might necessitate a different interleaver design compared to those with more random error distributions.

Ultimately, the efficacy of Turbo Codes is a function of these interdependent variables. By judiciously choosing and tuning these parameters, Turbo Codes can provide highly efficient and reliable communication even in adverse conditions. This adaptability and

efficiency make them a cornerstone of modern error correction and a topic of ongoing research in the field of digital communications.